

Overview

Graph AI is the specialization track for problems where relationships are first-class signals, not optional metadata. In classical tabular ML, rows are often treated as independent. In graph systems, value comes from dependencies across entities: transactions linked by shared devices, documents linked by citations, services linked by runtime calls, or users linked by interaction history.

GraphRAG extends retrieval-augmented generation by explicitly modeling entities and links before generation. Vector retrieval answers “what text looks similar?” Graph retrieval answers “what entities are connected, through which path, and with what provenance?” High-quality enterprise assistants usually need both.

A practical quality decomposition:

$$\text{Answer Quality} = f(\text{semantic recall}, \text{relational recall}, \text{grounded generation}, \text{citation fidelity})$$

When multi-hop relationship context matters, graph-aware retrieval often increases groundedness and reduces plausible-but-wrong outputs.

Core Concepts and Formal Definitions

Graph fundamentals

A graph is defined as:

$$G = (V, E)$$

where: - V is a set of nodes (entities), - E is a set of edges (relationships).

Applied choices that matter: - homogeneous vs heterogeneous node/edge types, - directed vs undirected relations, - weighted vs unweighted edges, - static vs temporal graphs.

Knowledge graph representation

Knowledge graphs typically store facts as triples:

$$(h, r, t)$$

where h is head entity, t is tail entity, and r is relation type. Examples: (ServiceA, depends_on, DatabaseB), (User123, purchased, Product98).

Graph ML tasks

Common tasks and business mappings: - node classification: risk score for an account node, - link prediction: likely fraud-ring connection, - community detection: cluster of related incidents, - graph-level prediction: classify an entire molecule or subnetwork.

GraphRAG Architecture Patterns

Indexing pipeline

A production indexing flow: 1. ingest source corpus and metadata, 2. chunk text and normalize identifiers, 3. extract entities and relations, 4. perform entity resolution and deduplication, 5. attach provenance

(document id, chunk id, timestamp), 6. build graph index, 7. build vector index for lexical/semantic fallback.

The Microsoft GraphRAG approach emphasizes configurable extraction and graph-aware indexing stages over one-shot embedding-only pipelines.

Query pipeline

Typical inference path: 1. query parsing and intent classification, 2. entity linking from query to graph, 3. neighborhood expansion (k-hop / constrained traversal), 4. optional vector retrieval for missing context, 5. hybrid reranking and context packing, 6. grounded generation with cited nodes/edges/chunks.

Retrieval fusion strategy

Recommended pattern: - run graph retrieval and vector retrieval in parallel, - normalize scores, - apply policy-aware reranking, - produce answer with explicit path/citation evidence.

This balances semantic recall with relational precision.

Decision Framework and Trade-offs

When GraphRAG is worth it

Use GraphRAG when: - user questions are multi-hop (“how is X impacted by Y?”), - provenance/path explanation is mandatory, - entity identity matters more than text similarity alone, - domain has stable entity schemas.

When vector-only RAG is sufficient

Use vector-only RAG when: - corpus is small/flat, - questions are mostly extractive single-hop, - latency budget is strict and relation signal is weak, - graph extraction quality is too low to trust.

Trade-off table

Dimension	Vector RAG	GraphRAG
Setup complexity	Lower	Higher
Multi-hop reasoning	Weak	Strong
Provenance path explainability	Moderate	High
Maintenance overhead	Moderate	High
Failure sensitivity to entity resolution	Low	High
Best-fit scenarios	FAQ, flat docs	Relationship-heavy workflows

Failure Modes and Reliability Patterns

Common failure modes

- entity linking mismatch (wrong node selected),
- relation extraction drift after schema/content changes,
- neighborhood explosion causing noisy prompts,
- stale graph snapshots causing outdated answers.

Guardrails

- confidence thresholds for extraction/linking,
- relation whitelist and schema constraints,
- max-hop and node-budget limits at query time,
- freshness SLAs and incremental re-index jobs,
- regression evals per high-risk query slice.

Evaluation slices

Track at least: - entity-linking precision/recall, - multi-hop retrieval success, - citation correctness, - end-to-end groundedness and task completion.

Frontier Case Studies & Exceptions

Case 1: Enterprise policy copilot

Scenario: compliance teams ask policy dependency questions spanning multiple documents and exception clauses.

Pattern: graph of policies, controls, exceptions, and owners + vector fallback for explanatory text.

Impact: reduced unsupported answers and improved audit traceability.

Exception: for a small policy corpus with minimal cross-linking, a high-quality vector RAG pipeline can be faster to ship.

Case 2: Fraud ring investigation

Scenario: fraud analysts need to detect coordinated abuse patterns.

Pattern: graph features from shared devices, IP ranges, and payment instruments; GraphRAG for analyst Q&A over investigation artifacts.

Impact: better recall of coordinated behavior than per-transaction models alone.

Exception: poor upstream identity resolution creates false edges and analyst fatigue.

Case 3: Service dependency assistant

Scenario: SRE teams need blast-radius and root-cause context during incidents.

Pattern: service dependency graph + incident timeline + runbook retrieval.

Impact: faster impact assessment and triage.

Exception: stale topology metadata causes incorrect dependency paths; enforce graph freshness checks before incident use.

Interview Questions & Answers

1. **Q:** What is GraphRAG?
A: A RAG architecture that augments semantic retrieval with graph extraction and graph-aware retrieval.
2. **Q:** Why not rely on vector retrieval only?
A: Vector similarity can miss relational paths and multi-hop dependencies.
3. **Q:** Define a knowledge graph triple.
A: (head entity, relation, tail entity).

4. **Q:** What is entity resolution?
A: Mapping different mentions/references to the same canonical entity node.
5. **Q:** What is multi-hop retrieval?
A: Traversing multiple edges to gather indirect but relevant context.
6. **Q:** Core graph ML tasks?
A: Node classification, link prediction, community detection, graph classification.
7. **Q:** Main GraphRAG production risk?
A: Entity/relation extraction errors propagating into wrong retrieval context.
8. **Q:** Why keep vector fallback in GraphRAG?
A: Graph coverage is rarely perfect; vector search recovers missing semantic context.
9. **Q:** How do you control graph retrieval noise?
A: Hop limits, node budgets, relation filtering, and reranking.
10. **Q:** Graph DB mandatory?
A: Not always, but usually needed for production scale and operational features.
11. **Q:** How does GraphRAG improve explainability?
A: It can show explicit relationship paths and cited source anchors.
12. **Q:** What metric checks citation quality?
A: Citation precision/groundedness against verified source spans.
13. **Q:** When should you avoid GraphRAG initially?
A: Small, flat corpora with mostly direct lookup questions.
14. **Q:** What is neighborhood expansion?
A: Collecting adjacent nodes/edges around linked entities during query time.
15. **Q:** Why is schema design important?
A: Ambiguous relation types reduce retrieval quality and increase hallucinated paths.
16. **Q:** How do you keep graphs fresh?
A: Incremental indexing, change-detection jobs, and freshness SLO monitoring.
17. **Q:** What is a hybrid retriever?
A: A retrieval layer combining graph and vector signals before generation.
18. **Q:** How do you evaluate GraphRAG end-to-end?
A: Measure retrieval quality (entity/link/path) plus answer groundedness and task success.
19. **Q:** Typical anti-pattern?
A: Adding graph complexity without a relation-heavy question set.
20. **Q:** One-line design rule?
A: Use GraphRAG when relationship structure is essential to answer correctness.

References

- Stanford CS224W: <https://web.stanford.edu/class/cs224w/>
- Microsoft GraphRAG overview: <https://microsoft.github.io/graphrag/index/overview/>
- LlamaIndex knowledge graph examples: https://docs.llamaindex.ai/en/stable/examples/index_structs/knowledge_gra